12주차 · 11/19(목)

HTML5 DOM 활용

DOM 구조 이해 및 요소 제어

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

12주차

학습 목표

1. DOM이 무엇인지 설명하고 HTML 문서와의 관계를 말할 수 있다.
2. querySelector로 요소를 찾고 내용·속성·클래스를 바꿀 수 있다.
3. textContent와 innerHTML의 차이와 보안 위험을 설명할 수 있다.
4. 요소를 만들어 붙이고 지울 수 있다.
5. querySelector가 null을 돌려주는 원인을 진단할 수 있다.

12주차

핵심 키워드

DOM · querySelector · textContent · innerHTML · XSS · classList · dataset · createElement

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

12주차 · 1

DOM — 브라우저가 만든 문서의 지도

브라우저는 HTML 파일을 읽어 객체들의 나무(tree) 로 바꿔 메모리에 올립니다.

DOM — 브라우저가 만든 문서의 지도

1. 이것이 DOM(Document Object Model) 입니다.
2. JavaScript는 이 나무를 타고 다니며 원하는 가지를 찾아 바꿉니다.

HTML이 나무가 되는 모습

```
<!-- HTML 파일 -->
<body>
  <h1 id="title">제목</h1>
  <ul class="list">
    <li>하나</li>
    <li>둘</li>
  </ul>
</body>

/* 브라우저가 만든 DOM 나무
document
└─ html
   └─ body
      ├─ h1#title
      │  └─ "제목"          ← 텍스트 노드
      └─ ul.list
         ├─ li → "하나"
         └─ li → "둘"       */
```

꼭 구분해야 할 것

정리

1. **HTML 파일**
   은 서버에 있는 글자 뭉치입니다 — 바뀌지 않습니다.

2. **DOM**
   은 브라우저 메모리 안의 객체 나무입니다 — JavaScript가 바꾸는 것은 이쪽입니다.

3. **그래서 JS로 화면을 바꿔도 새로고침하면 원래대로 돌아옵니다. 원본 파일은 그대로니까요.**

12주차 · 2

요소 찾기

querySelector — CSS 선택자를 그대로 씁니다

```
document.querySelector('#title');      // id — 하나
document.querySelector('.list');       // class — 첫 번째 하나
document.querySelector('li');          // 태그 — 첫 번째 하나
document.querySelector('.list > li');  // 복잡한 선택자도 그대로

document.querySelectorAll('li');       // 전부 (NodeList)

// 못 찾으면 null을 돌려줍니다 — 이게 오류의 시작점입니다
const el = document.querySelector('#없는아이디');
console.log(el);            // null
el.textContent = '안녕';    // ✗ TypeError: Cannot set properties of null
```

요소 찾기

1. getElementById · getElementsByClassName 같은 옛 방법도 있지만, querySelector 하나만 알면 됩니다.
2. CSS에서 쓰던 선택자를 그대로 쓸 수 있어 외울 것이 없습니다.

① 선택자 오타 ② script가 너무 일찍 실행됨

```
<!-- ✗ head에 있는 script는 body보다 먼저 실행됩니다 -->
<head>
  <script>
    document.querySelector('#btn');   // null — 아직 btn이 안 만들어졌다
  </script>
</head>
<body>
  <button id="btn">버튼</button>
</body>

<!-- ✓ 해결 1: script를 </body> 바로 앞에 -->
<!-- ✓ 해결 2: defer 속성 — <script src="app.js" defer></script> -->
<!-- ✓ 해결 3: DOM이 다 만들어진 뒤 실행 -->
<script>
  document.addEventListener('DOMContentLoaded', () => {
    document.querySelector('#btn');   // 이제 있습니다
  });
</script>
```

querySelectorAll은 배열이 아닙니다

```
const items = document.querySelectorAll('li');

items.forEach(li => console.log(li.textContent));   // ✓ forEach는 됩니다
items.map(li => li.textContent);                    // ✗ map은 없습니다

const arr = [...items];          // 배열로 바꾸면 전부 쓸 수 있습니다
arr.map(li => li.textContent); // ✓
```

12주차 · 3

내용과 모양 바꾸기

글자 바꾸기

```
const title = document.querySelector('#title');

title.textContent = '바뀐 제목';        // 글자만 — 안전 ✓
title.innerHTML = '<em>기울임</em>';    // HTML로 해석 — 조심

// 읽기도 됩니다
console.log(title.textContent);
```

innerHTML을 조심해야 하는 이유 (XSS)

정리

1. **innerHTML 은 넣은 문자열을 HTML로 해석합니다. 그래서 사용자가 쓴 글을 그대로 넣으면 그 안의 <script> 나 onerror 가 실제로 실행됩니다.**

2. **이것을 XSS(Cross-Site Scripting) 공격이라 하며, 남의 로그인 정보를 훔치는 데 쓰입니다.**

3. **원칙: 사용자가 입력한 것은 반드시 textContent**
로 넣습니다. innerHTML 은 내가 직접 쓴 문자열에만 씁니다.

입력과 값

```
const input = document.querySelector('#name');

input.value                         // 입력칸의 현재 값 (항상 문자열!)
input.value = '기본값';
Number(input.value)                 // 계산하려면 변환 필수

const check = document.querySelector('#agree');
check.checked                       // 체크박스는 checked (true/false)
```

스타일 — 클래스로 바꿉니다

```
const box = document.querySelector('.box');

// ✗ 이렇게 직접 만지지 마세요 — 스타일이 JS 안에 흩어집니다
box.style.background = 'red';
box.style.padding = '20px';

// ✓ 스타일은 CSS에 두고, JS는 클래스만 바꿉니다
box.classList.add('is-active');
box.classList.remove('is-active');
box.classList.toggle('is-active');          // 있으면 빼고 없으면 넣기
box.classList.toggle('is-active', true);    // 강제로 켜기
box.classList.contains('is-active');        // 있는지 확인
```

속성 다루기

```
img.src = 'new.jpg';
link.href = 'https://example.com';

el.setAttribute('data-id', '42');
el.getAttribute('data-id');       // '42'
el.dataset.id;                    // '42'  ← data-* 는 dataset으로 더 편하게
el.removeAttribute('disabled');

// 보이기/숨기기 — hidden 속성이 가장 간단합니다
panel.hidden = true;
panel.hidden = false;
/* CSS에 [hidden] { display: none !important } 를 함께 두세요.
   다른 display 규칙이 hidden을 이겨 버리는 일이 실제로 자주 있습니다. */
```

12주차 · 4

요소 만들고 붙이기

createElement + append

```
const li = document.createElement('li');   // 만들고
li.textContent = '새 항목';                  // 채우고
li.className = 'item';
document.querySelector('ul').append(li);   // 붙인다

// 여러 개
['하나', '둘', '셋'].forEach(text => {
  const li = document.createElement('li');
  li.textContent = text;
  ul.append(li);
});

// 지우기
li.remove();                // 자기 자신 제거
ul.textContent = '';        // 자식 전부 비우기 (가장 간단한 방법)
```

요소 만들고 붙이기

메서드	어디에
append(el)	자식의 맨 뒤
prepend(el)	자식의 맨 앞
before(el) / after(el)	자기 앞 / 뒤 (형제로)
replaceWith(el)	자기 자리를 대체
remove()	자기 자신 제거

12주차

실습

이 시간의 실습 3개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 버튼을 누르면 h1의 글자가 바뀌게 하세요. HTML은 JavaScript로 만들어도 되고, document.body.innerHTML로 먼저 넣어도 됩니다.

힌트

querySelector로 찾고 addEventListener로 걸고 textContent로 바꿉니다.

실습 1 모범답안

모범답안

```
document.body.innerHTML = '<h1 id="title">처음 제목</h1><button id="btn">바꾸기</button>';

document.querySelector('#btn').addEventListener('click', () => {
  document.querySelector('#title').textContent = '바뀐 제목! ';
});
```

실습 2 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 목록을 JavaScript로 만들어 화면에 붙이세요. 배열 ["HTML","CSS","JavaScript"]를 돌면서 li를 만들어 ul에 넣습니다. 사용자 입력이 아니어도 textContent를 쓰세요.

힌트

createElement로 만들고, textContent로 채우고, append로 붙입니다.

실습 2 모범답안

모범답안

```
document.body.innerHTML = '<h2>배우는 것</h2><ul id="list"></ul>';

const items = ['HTML', 'CSS', 'JavaScript'];
const ul = document.querySelector('#list');

items.forEach((text) => {
  const li = document.createElement('li');
  li.textContent = text;
  ul.append(li);
});
```

실습 3 · 응용

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 버튼을 누르면 상자에 클래스가 붙었다 떨어지게 하세요. 스타일은 CSS에 두고 JavaScript는 클래스만 토글해야 합니다. 지금 상태를 화면 글자로도 표시하세요.

힌트

classList.toggle()은 붙였는지 여부를 true/false로 돌려줍니다. 그 값을 그대로 쓰면 편합니다.

실습 3 모범답안

● 모범답안

```
document.body.innerHTML = `
  <style>
    .box { padding: 24px; border: 2px solid #cbd5e1; border-radius: 12px; transition: .2s; }
    .box.is-on { background: #4338ca; color: #fff; border-color: #4338ca; }
  </style>
  <div class="box" id="box">상자</div>
  <button id="btn">켜기 / 끄기</button>
  <p id="state"></p>
`;

const box = document.querySelector('#box');
const state = document.querySelector('#state');

document.querySelector('#btn').addEventListener('click', () => {
  const on = box.classList.toggle('is-on');
  state.textContent = on ? '켜짐' : '꺼짐';
});
```

12주차 마무리

12주차 마무리

HTML5 DOM 활용 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

- ✓ 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시
- ✓ 실습 3개를 실습장에서 「확인」 까지 — 「실습했어요」 체크
- ✓ 퀴즈 10 「퀴즈 10 — HTML5 DOM 활용」 응시 (성적 미반영 · 학습 확인용)
- ✓ 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

13주차 · 이벤트 처리 기초 및 응용